

Étude Scientifique de l'Expressivité du DSL PlanningSpec

Projet M2 Recherche – DSL de planification déclarative

18 avril 2026

Table des matières

1	Introduction	3
2	Cadre scientifique et positionnement	3
2.1	Références de base	3
2.2	Unité d'analyse	3
3	Notion d'expressivité : définitions	3
4	Modèle formel global de PlanningSpec	4
5	Contribution de chaque contrainte à l'expressivité	5
5.1	Contraintes dures	5
5.2	Préférences (soft constraints)	5
6	Traduction MiniZinc et logique mathématique	5
7	Formalisation détaillée des contraintes et préférences	6
7.1	Contraintes dures	6
7.1.1	cardinality_per_activity	6
7.1.2	resource_exclusivity	6
7.1.3	fixed_assignment et forbidden_assignment	6
7.1.4	temporal_precedence et instance_precedence	7
7.1.5	time_window	7
7.1.6	mandatory_roles	7
7.1.7	required_resource	7
7.2	Préférences (soft constraints)	7
7.2.1	avoid_participation_on_date	7
7.2.2	max_per_scope	7
7.2.3	preferred_resource	8
7.3	Objectif global	8
8	Démonstrations sur des cas de la littérature	8
8.1	Cas A – JSSP (article de synthèse)	8
8.2	Cas B – RCPSP (dépendances + ressources renouvelables)	10
8.3	Cas C – Planification académique à rôles	12

9	Discussion scientifique	14
9.1	Ce qui est prouvé par les démonstrations	14
9.2	Limites actuelles	14
9.3	Menaces à la validité	14
10	Conclusion	15

1 Introduction

La modélisation déclarative des problèmes de planification est reconnue comme un verrou méthodologique : la puissance des solveurs ne suffit pas si la spécification du problème reste difficile à formaliser pour les experts métier [3, 5]. Les travaux récents confirment cet enjeu, en particulier dans les approches NL-to-CP où la qualité de la modélisation demeure le facteur limitant [6, 8, 9].

Ce rapport répond à la question suivante :

*Le langage **PlanningSpec** est-il suffisamment expressif pour encoder, de manière rigoureuse, des classes de problèmes de planification issues de la littérature ?*

Notre contribution dans ce document est de :

- C1.** définir formellement la notion d’expressivité utilisée pour l’évaluation ;
- C2.** caractériser la contribution de chaque contrainte/préférence du DSL à cette expressivité ;
- C3.** démontrer l’encodage de cas représentatifs de la littérature (JSSP, RCPSP, planification académique) ;
- C4.** fournir les fichiers `.planning` reproductibles correspondant à ces démonstrations.

2 Cadre scientifique et positionnement

2.1 Références de base

Nous nous appuyons sur trois blocs de littérature :

- **Langages de modélisation CP** : MiniZinc comme référence solver-agnostic [4].
- **Problèmes de planification** : JSSP et sa taxonomie de contraintes/objectifs [1], complexité NP-difficile [2], et variantes RCPSP rapportées dans la revue systématique interne.
- **Assistants de modélisation récents** : Text2Zinc, ConstraintLLM, MCP-Solver, qui motivent un DSL explicite et validable [6, 5, 9].

2.2 Unité d’analyse

L’unité d’analyse est le couple :

(Grammaire `PlanningSpec`, Compilation `MiniZinc`)

L’expressivité ne se réduit donc pas à la syntaxe ; elle inclut la capacité de traduction en contraintes exécutables.

3 Notion d’expressivité : définitions

Définition 1 (Expressivité syntaxique). *Un DSL est syntaxiquement expressif pour une famille de problèmes si chaque concept métier critique (temps, ressources, précédences, capacités, préférences) dispose d’un construct explicite et non ambigu.*

Définition 2 (Expressivité sémantique). *Un DSL est sémantiquement expressif si ses constructs peuvent être compilés en contraintes logiques/mathématiques préservant l'intention du modèle.*

Définition 3 (Expressivité compositionnelle). *Un DSL est compositionnellement expressif si plusieurs contraintes peuvent être combinées sans contradiction systématique du formalisme et sans perdre la capacité de résolution.*

Définition 4 (Expressivité opérationnelle). *Un DSL est opérationnellement expressif s'il permet de produire un modèle effectivement résoluble par un solveur générique (ici Mini-Zinc + solveur CP).*

4 Modèle formel global de PlanningSpec

On note :

$\mathcal{A}$	: ensemble des instances d'activités
$\mathcal{R}$	: ensemble des ressources
$\mathcal{K}$	: ensemble des rôles
H	: horizon discret (slots)
d_i	: durée de l'activité $i \in \mathcal{A}$

Variables :

$s_i \in \{1, \dots, H\}$	(début)
$x_{i,r} \in \{0, 1\}$	(ressource r affectée à i)
$y_{i,k,r} \in \{0, 1\}$	(ressource r affectée au rôle k de i)

Contraintes structurelles compilées :

1. non-dépassement journalier des activités ;
2. non-chevauchement d'une même ressource ;
3. cohérence rôle–type de ressource ;
4. cohérence `role_assignment` $\Rightarrow$ `assignment`.

5 Contribution de chaque contrainte à l'expressivité

5.1 Contraintes dures

Contrainte DSL	Rôle scientifique	Contribution à l'expressivité
<code>cardinality_per_activity</code>	Encodage d'arité de participation (ressource/rôle)	Permet les exigences quantitatives (ex. 1 président, 2 opérateurs)
<code>resource_exclusivity</code>	Capacité locale slot/jour par type de ressource	Modélise les ressources renouvelables et conflits d'utilisation
<code>fixed_assignment</code>	Affectation imposée	Représente les obligations organisationnelles fortes
<code>forbidden_assignment</code>	Affectation interdite	Exprime incompatibilités réglementaires/techniques
<code>temporal_precedence</code>	Ordre global inter-types d'activités	Capture contraintes de phases et dépendances macro
<code>instance_precedence</code>	Ordre fin entre instances	Capture les graphes de précedence de type JSSP/RCPSP
<code>time_window</code>	Fenêtre temporelle bornée	Exprime dates de disponibilité et fenêtres contractuelles
<code>mandatory_roles</code>	Rôles obligatoires	Garantit la complétude structurale des activités complexes
<code>required_resource</code>	Ressource explicitement requise	Permet routage machine/opération, affectations matérielles imposées

5.2 Préférences (soft constraints)

Préférence DSL	Rôle scientifique	Contribution à l'expressivité
<code>avoid_participation_on_date</code>	Indisponibilité souple datée	Introduit flexibilité sans rendre le modèle infaisable
<code>max_per_scope</code>	Plafonnement souple par jour/slot	Permet équilibrage de charge et robustesse organisationnelle
<code>preferred_resource</code>	Préférence d'affectation	Modélise qualité de solution et satisfaction métier

6 Traduction MiniZinc et logique mathématique

La compilation suit une logique CP classique [4, 3]. Exemples centraux :

Cardinalité par activité et type τ

$$\forall i \in \mathcal{A}(a) : \quad m \leq \sum_{r \in \mathcal{R}_\tau} x_{i,r} \leq M$$

Exclusivité ressource (scope slot)

$$\forall r, \forall t : \sum_{i \in \mathcal{A}(a)} \mathbf{1}[x_{i,r} = 1 \wedge s_i \leq t < s_i + d_i] \leq M$$

Précédence d'instances

$$s_{i_b} + d_{i_b} \leq s_{i_a}$$

Préférences agrégées

$$\text{penalty} = \sum_{\ell} P_{\ell}, \quad \text{solve minimize penalty}$$

(ou satisfy en absence de préférences).

7 Formalisation détaillée des contraintes et préférences

Cette section explicite, pour chaque construct du DSL, son intérêt scientifique, sa nécessité pratique, et sa formulation mathématique.

7.1 Contraintes dures

7.1.1 cardinality_per_activity

Pourquoi c'est nécessaire. Les problèmes de planification imposent souvent des exigences quantitatives minimales et maximales par activité (nombre d'opérateurs, nombre de rôles, etc.), point central dans les formulations RCPSP/JSSP étendues [1, 10].

Formulation. Pour une activité a , une instance $i \in \mathcal{A}(a)$, une borne $[m, M]$:

$$m \leq \sum_{r \in \mathcal{R}_{\tau}} x_{i,r} \leq M$$

ou, pour un rôle k :

$$m \leq \sum_{r \in \mathcal{R}_{\text{type}(k)}} y_{i,k,r} \leq M.$$

7.1.2 resource_exclusivity

Pourquoi c'est nécessaire. La capacité finie des ressources renouvelables est la contrainte structurante des modèles de scheduling : une machine/ressource ne peut exécuter plusieurs tâches concurrentes au-delà de sa capacité [3, 1].

Formulation (scope slot).

$$\forall r \in \mathcal{R}_{\tau}, \forall t \in \{1, \dots, H\} : \sum_{i \in \mathcal{A}(a)} \mathbf{1}[x_{i,r} = 1 \wedge s_i \leq t < s_i + d_i] \leq M.$$

7.1.3 fixed_assignment et forbidden_assignment

Pourquoi c'est nécessaire. Les cas réels imposent des obligations d'affectation (certification, habilitation, disponibilité contractuelle) et des incompatibilités explicites [10, 7].

Formulation.

$$\text{fixed: } x_{i,r} = 1 \text{ (et } y_{i,k,r} = 1 \text{ si rôle),} \quad \text{forbidden: } y_{i,k,r} = 0.$$

7.1.4 temporal_precedence et instance_precedence

Pourquoi c'est nécessaire. Les contraintes de précédence modélisent la logique technologique et organisationnelle du flux de production/projet [2, 1].

Formulation.

$$\text{temporal_precedence: } \forall i \in \mathcal{A}(a_b), \forall j \in \mathcal{A}(a_a), s_i + d_i \leq s_j$$

$$\text{instance_precedence: } s_{i_b} + d_{i_b} \leq s_{i_a}.$$

7.1.5 time_window

Pourquoi c'est nécessaire. Les fenêtres calendaires représentent disponibilités, jalons et contraintes d'ouverture atelier/salle [10].

Formulation.

$$L \leq s_i, \quad s_i + d_i - 1 \leq U.$$

7.1.6 mandatory_roles

Pourquoi c'est nécessaire. Dans les activités multi-rôles (maintenance spécialisée, jury académique), la faisabilité métier exige la couverture de chaque rôle clé.

Formulation.

$$\forall i \in \mathcal{A}(a), \forall k \in \mathcal{K}(a) : \sum_{r \in \mathcal{R}} y_{i,k,r} \geq 1.$$

7.1.7 required_resource

Pourquoi c'est nécessaire. Permet d'encoder explicitement les opérations qui requièrent une ressource déterminée (routage machine/opération), typique des problèmes job-shop.

Formulation.

$$x_{i,r} = 1.$$

7.2 Préférences (soft constraints)

7.2.1 avoid_participation_on_date

Pourquoi c'est nécessaire. Les indisponibilités souhaitées (et non strictes) sont fréquentes. Les modéliser en pénalité améliore l'acceptabilité du planning sans casser la faisabilité [10].

Formulation.

$$P = w \cdot \mathbf{1} \left[\sum_{i \in \mathcal{A}} \mathbf{1}[x_{i,r} = 1 \wedge \text{dayOf}(s_i) = q] > 0 \right].$$

7.2.2 max_per_scope

Pourquoi c'est nécessaire. Cette préférence sert au lissage de charge (workload balancing) et à la robustesse opérationnelle.

Formulation (slot).

$$P = \sum_{r,t} w \cdot \max \left(0, \sum_{i \in \mathcal{A}(a)} \mathbf{1}[x_{i,r} = 1 \wedge s_i \leq t < s_i + d_i] - M \right).$$

7.2.3 preferred_resource

Pourquoi c'est nécessaire. Représente les choix de qualité de service/expérience locale tout en restant flexible.

Formulation.

$$P = \begin{cases} w \cdot \mathbf{1}[y_{i,k,r} = 0], & \text{si rôle explicite} \\ w \cdot \mathbf{1}[x_{i,r} = 0], & \text{sinon} \end{cases}$$

7.3 Objectif global

Le solveur minimise la pénalité agrégée :

$$\min \sum_{\ell} P_{\ell}$$

ou recherche une solution faisable (**satisfy**) s'il n'existe aucune préférence. Ce choix est cohérent avec une stratégie de planification d'abord faisable puis améliorable, courante en CP appliquée [4, 5].

8 Démonstrations sur des cas de la littérature

8.1 Cas A – JSSP (article de synthèse)

Le JSSP est caractérisé par des opérations ordonnées par job, des ressources machine unitaires, et un horizon commun [1, 2]. Notre encodage utilise :

- `instance_precedence` pour les routes d'opérations ;
- `required_resource` pour la machine de chaque opération ;
- `resource_exclusivity` + non-chevauchement global pour la capacité machine ;
- `cardinality_per_activity` pour imposer 1 machine/opération.

Fichier de démonstration `cases/01_jssp_article_case.planning`

```
{
  "time": {
    "days": ["D1", "D2", "D3"],
    "slotsPerDay": 8
  },
  "activities": {
    "J1_01": { "count": 1, "duration": 2 },
    "J1_02": { "count": 1, "duration": 2 },
    "J1_03": { "count": 1, "duration": 1 },
    "J2_01": { "count": 1, "duration": 2 },
    "J2_02": { "count": 1, "duration": 1 },
    "J2_03": { "count": 1, "duration": 2 },
    "J3_01": { "count": 1, "duration": 1 },
    "J3_02": { "count": 1, "duration": 2 },
    "J3_03": { "count": 1, "duration": 2 }
  },
  "resources": {
    "Machine": ["M1", "M2", "M3"]
  }
}
```



```

},
"roles": {},
"constraints": [
  { "type": "cardinality_per_activity", "activity": "J1_01", "
    target": "Machine", "min": 1, "max": 1 },
  { "type": "cardinality_per_activity", "activity": "J1_02", "
    target": "Machine", "min": 1, "max": 1 },
  { "type": "cardinality_per_activity", "activity": "J1_03", "
    target": "Machine", "min": 1, "max": 1 },
  { "type": "cardinality_per_activity", "activity": "J2_01", "
    target": "Machine", "min": 1, "max": 1 },
  { "type": "cardinality_per_activity", "activity": "J2_02", "
    target": "Machine", "min": 1, "max": 1 },
  { "type": "cardinality_per_activity", "activity": "J2_03", "
    target": "Machine", "min": 1, "max": 1 },
  { "type": "cardinality_per_activity", "activity": "J3_01", "
    target": "Machine", "min": 1, "max": 1 },
  { "type": "cardinality_per_activity", "activity": "J3_02", "
    target": "Machine", "min": 1, "max": 1 },
  { "type": "cardinality_per_activity", "activity": "J3_03", "
    target": "Machine", "min": 1, "max": 1 },

  { "type": "required_resource", "activityInstance": "J1_01_1", "
    resource": "M1" },
  { "type": "required_resource", "activityInstance": "J1_02_1", "
    resource": "M2" },
  { "type": "required_resource", "activityInstance": "J1_03_1", "
    resource": "M3" },
  { "type": "required_resource", "activityInstance": "J2_01_1", "
    resource": "M1" },
  { "type": "required_resource", "activityInstance": "J2_02_1", "
    resource": "M3" },
  { "type": "required_resource", "activityInstance": "J2_03_1", "
    resource": "M2" },
  { "type": "required_resource", "activityInstance": "J3_01_1", "
    resource": "M2" },
  { "type": "required_resource", "activityInstance": "J3_02_1", "
    resource": "M1" },
  { "type": "required_resource", "activityInstance": "J3_03_1", "
    resource": "M3" },

  { "type": "instance_precedence", "beforeActivityInstance": "
    J1_01_1", "afterActivityInstance": "J1_02_1" },
  { "type": "instance_precedence", "beforeActivityInstance": "
    J1_02_1", "afterActivityInstance": "J1_03_1" },
  { "type": "instance_precedence", "beforeActivityInstance": "
    J2_01_1", "afterActivityInstance": "J2_02_1" },
  { "type": "instance_precedence", "beforeActivityInstance": "
    J2_02_1", "afterActivityInstance": "J2_03_1" },
  { "type": "instance_precedence", "beforeActivityInstance": "
    J3_01_1", "afterActivityInstance": "J3_02_1" },

```

```

{ "type": "instance_precedence", "beforeActivityInstance": "
  J3_02_1", "afterActivityInstance": "J3_03_1" },

{ "type": "resource_exclusivity", "resourceType": "Machine", "
  activity": "J1_01", "scope": "slot", "max": 1 },
{ "type": "resource_exclusivity", "resourceType": "Machine", "
  activity": "J1_02", "scope": "slot", "max": 1 },
{ "type": "resource_exclusivity", "resourceType": "Machine", "
  activity": "J1_03", "scope": "slot", "max": 1 },
{ "type": "resource_exclusivity", "resourceType": "Machine", "
  activity": "J2_01", "scope": "slot", "max": 1 },
{ "type": "resource_exclusivity", "resourceType": "Machine", "
  activity": "J2_02", "scope": "slot", "max": 1 },
{ "type": "resource_exclusivity", "resourceType": "Machine", "
  activity": "J2_03", "scope": "slot", "max": 1 },
{ "type": "resource_exclusivity", "resourceType": "Machine", "
  activity": "J3_01", "scope": "slot", "max": 1 },
{ "type": "resource_exclusivity", "resourceType": "Machine", "
  activity": "J3_02", "scope": "slot", "max": 1 },
{ "type": "resource_exclusivity", "resourceType": "Machine", "
  activity": "J3_03", "scope": "slot", "max": 1 }
],
"preferences": []
}

```

Proposition 1. *Sous l'hypothèse d'une route machine fixée par opération, le fichier JSSP fourni réalise un encodage CP cohérent des contraintes structurelles usuelles du JSSP (précédence intra-job, non-conflit machine, durée).*

8.2 Cas B – RCPSP (dépendances + ressources renouvelables)

Dans l'esprit des formulations RCPSP (et des encodages SAT/SMT recensés dans la revue interne), nous modélisons :

- un graphe de précédence de projet ;
- des demandes de ressources renouvelables (workers, grue) ;
- des fenêtres temporelles partielles ;
- une préférence de lissage de charge.

Fichier de démonstration cases/02_rcpsp_article_case.planning

```

{
  "time": {
    "days": ["D1", "D2", "D3", "D4", "D5"],
    "slotsPerDay": 6
  },
  "activities": {
    "Start": { "count": 1, "duration": 1 },
    "A": { "count": 1, "duration": 3 },
    "B": { "count": 1, "duration": 2 },
    "C": { "count": 1, "duration": 2 },

```

```

    "D": { "count": 1, "duration": 2 },
    "E": { "count": 1, "duration": 1 },
    "Finish": { "count": 1, "duration": 1 }
  },
  "resources": {
    "Worker": ["W1", "W2", "W3"],
    "Crane": ["C1"]
  },
  "roles": {},
  "constraints": [
    { "type": "cardinality_per_activity", "activity": "A", "target": "Worker", "min": 2, "max": 2 },
    { "type": "cardinality_per_activity", "activity": "B", "target": "Worker", "min": 1, "max": 1 },
    { "type": "cardinality_per_activity", "activity": "C", "target": "Worker", "min": 1, "max": 1 },
    { "type": "cardinality_per_activity", "activity": "D", "target": "Worker", "min": 2, "max": 2 },
    { "type": "cardinality_per_activity", "activity": "E", "target": "Worker", "min": 1, "max": 1 },

    { "type": "cardinality_per_activity", "activity": "A", "target": "Crane", "min": 1, "max": 1 },
    { "type": "cardinality_per_activity", "activity": "D", "target": "Crane", "min": 1, "max": 1 },

    { "type": "instance_precedence", "beforeActivityInstance": "Start_1", "afterActivityInstance": "A_1" },
    { "type": "instance_precedence", "beforeActivityInstance": "Start_1", "afterActivityInstance": "B_1" },
    { "type": "instance_precedence", "beforeActivityInstance": "A_1", "afterActivityInstance": "C_1" },
    { "type": "instance_precedence", "beforeActivityInstance": "A_1", "afterActivityInstance": "D_1" },
    { "type": "instance_precedence", "beforeActivityInstance": "B_1", "afterActivityInstance": "D_1" },
    { "type": "instance_precedence", "beforeActivityInstance": "C_1", "afterActivityInstance": "E_1" },
    { "type": "instance_precedence", "beforeActivityInstance": "D_1", "afterActivityInstance": "E_1" },
    { "type": "instance_precedence", "beforeActivityInstance": "E_1", "afterActivityInstance": "Finish_1" },

    { "type": "time_window", "activityInstance": "A_1", "minSlot": 2, "maxSlot": 20 },
    { "type": "time_window", "activityInstance": "D_1", "minSlot": 4, "maxSlot": 26 },

    { "type": "resource_exclusivity", "resourceType": "Crane", "activity": "A", "scope": "slot", "max": 1 },
    { "type": "resource_exclusivity", "resourceType": "Crane", "

```

```

        activity": "D", "scope": "slot", "max": 1 },

    { "type": "resource_exclusivity", "resourceType": "Worker", "
      activity": "A", "scope": "slot", "max": 1 },
    { "type": "resource_exclusivity", "resourceType": "Worker", "
      activity": "B", "scope": "slot", "max": 1 },
    { "type": "resource_exclusivity", "resourceType": "Worker", "
      activity": "C", "scope": "slot", "max": 1 },
    { "type": "resource_exclusivity", "resourceType": "Worker", "
      activity": "D", "scope": "slot", "max": 1 },
    { "type": "resource_exclusivity", "resourceType": "Worker", "
      activity": "E", "scope": "slot", "max": 1 }
  ],
  "preferences": [
    {
      "type": "max_per_scope",
      "resourceType": "Worker",
      "activity": "D",
      "scope": "day",
      "max": 1,
      "weight": 3
    }
  ]
}

```

Proposition 2. *Le cas RCPSP encode simultanément précédences, capacités et fenêtres, ce qui montre la capacité compositionnelle du DSL sur des contraintes hétérogènes.*

8.3 Cas C – Planification académique à rôles

Ce cas illustre l’expressivité spécifique du DSL pour des problèmes institutionnels (jurys/soutenances) :

- rôles obligatoires (`mandatory_roles`);
- cardinalités par rôle;
- exclusivité enseignant/salle;
- préférences personnelles (jour à éviter, ressource préférée).

Fichier de démonstration `cases/03_academic_planning_case.planning`

```

{
  "time": {
    "days": ["Lundi", "Mardi", "Mercredi"],
    "slotsPerDay": 8
  },
  "activities": {
    "Soutenance": { "count": 6, "duration": 1 },
    "ReunionJury": { "count": 2, "duration": 2 }
  },
  "resources": {
    "Teacher": ["T1", "T2", "T3", "T4", "T5"],

```

```

    "Room": ["R1", "R2"],
    "Student": ["S1", "S2", "S3", "S4", "S5", "S6"]
  },
  "roles": {
    "Soutenance": {
      "President": "Teacher",
      "Rapporteur": "Teacher",
      "Encadreur": "Teacher",
      "Salle": "Room"
    },
    "ReunionJury": {
      "President": "Teacher",
      "Salle": "Room"
    }
  },
  "constraints": [
    { "type": "mandatory_roles", "activity": "Soutenance" },
    { "type": "mandatory_roles", "activity": "ReunionJury" },

    { "type": "cardinality_per_activity", "activity": "Soutenance",
      "role": "President", "min": 1, "max": 1 },
    { "type": "cardinality_per_activity", "activity": "Soutenance",
      "role": "Rapporteur", "min": 1, "max": 1 },
    { "type": "cardinality_per_activity", "activity": "Soutenance",
      "role": "Encadreur", "min": 1, "max": 1 },
    { "type": "cardinality_per_activity", "activity": "Soutenance",
      "role": "Salle", "min": 1, "max": 1 },

    { "type": "cardinality_per_activity", "activity": "ReunionJury",
      "role": "President", "min": 1, "max": 1 },
    { "type": "cardinality_per_activity", "activity": "ReunionJury",
      "role": "Salle", "min": 1, "max": 1 },

    { "type": "resource_exclusivity", "resourceType": "Teacher", "
      activity": "Soutenance", "scope": "slot", "max": 1 },
    { "type": "resource_exclusivity", "resourceType": "Teacher", "
      activity": "ReunionJury", "scope": "slot", "max": 1 },
    { "type": "resource_exclusivity", "resourceType": "Room", "
      activity": "Soutenance", "scope": "slot", "max": 1 },
    { "type": "resource_exclusivity", "resourceType": "Room", "
      activity": "ReunionJury", "scope": "slot", "max": 1 },

    { "type": "time_window", "activityInstance": "ReunionJury_1", "
      minSlot": 1, "maxSlot": 8 },
    { "type": "time_window", "activityInstance": "ReunionJury_2", "
      minSlot": 9, "maxSlot": 16 },
    { "type": "temporal_precedence", "beforeActivity": "ReunionJury",
      "afterActivity": "Soutenance" }
  ],
  "preferences": [
    { "type": "avoid_participation_on_date", "resource": "T1", "

```

```

    date": "Mardi", "weight": 4 },
  { "type": "preferred_resource", "activityInstance": "
    Soutenance_1", "role": "President", "resource": "T2", "
    weight": 3 },
  { "type": "preferred_resource", "activityInstance": "
    Soutenance_2", "role": "President", "resource": "T3", "
    weight": 2 },
  { "type": "max_per_scope", "resourceType": "Teacher", "activity
    ": "Soutenance", "scope": "day", "max": 2, "weight": 2 }
]
}

```

Proposition 3. *La combinaison contraintes dures + préférences pondérées permet d’exprimer la faisabilité institutionnelle et la qualité d’usage dans un même modèle.*

9 Discussion scientifique

9.1 Ce qui est prouvé par les démonstrations

Les trois cas montrent que `PlanningSpec` couvre explicitement :

1. précédence et temporalité ;
2. capacité/partage de ressources ;
3. affectations forcées/interdites ;
4. séparation dur/souple et optimisation par pénalité.

Ces dimensions correspondent aux briques centrales des formulations de planification combinatoire [1, 3].

9.2 Limites actuelles

- objectif `makespan` non activé par défaut (choix pragmatique pour limiter l’explosion combinatoire) ;
- `temporal_precedence` est global (all-to-all) et peut être trop fort selon le domaine ;
- certaines vérifications sémantiques avancées (cycles, cohérence référentielle exhaustive) peuvent être renforcées.

9.3 Menaces à la validité

- les cas d’étude sont représentatifs mais de taille réduite ;
- certaines références récentes (LLM+CP) restent prépublications ;
- l’évaluation porte sur l’expressivité et non sur un benchmark de performance exhaustif.

10 Conclusion

Au sens des définitions proposées (syntaxique, sémantique, compositionnelle, opérationnelle), **PlanningSpec** est expressif pour une large classe de problèmes de planification discrète à ressources contraintes. Les encodages fournis montrent la transposition effective de cas de littérature vers des modèles exécutables.

La suite scientifique naturelle est :

1. ajouter des objectifs multi-critères explicites (e.g., makespan + pénalités),
2. enrichir la validation statique du DSL,
3. conduire une campagne expérimentale comparative sur jeux d’instances standard.

Références

- [1] Majid Abdolrazzagah-Nezhad and Salwani Abdullah. Job shop scheduling : Classification, constraints and objective functions. *International Journal of Computer, Electrical, Automation, Control and Information Engineering*, 11(4), 2017.
- [2] Michael R. Garey, David S. Johnson, and Ravi Sethi. The complexity of flowshop and jobshop scheduling. *Mathematics of Operations Research*, 1(2) :117–129, 1976.
- [3] Kim Marriott and Peter J. Stuckey. *Programming with Constraints : An Introduction*. MIT Press, 1998.
- [4] Nicholas Nethercote, Peter J. Stuckey, Ralph Becket, Sebastian Brand, Gregory J. Duck, and Guido Tack. Minizinc : Towards a standard cp modelling language. In *Principles and Practice of Constraint Programming (CP)*. Springer, 2007.
- [5] Weichun Shi, Minghao Liu, Wanting Zhang, Langchen Shi, Fuqi Jia, Feifei Ma, and Jian Zhang. Constraintllm : A neuro-symbolic framework for industrial-level constraint programming. In *Proceedings of EMNLP*, pages 15999–16019, 2025.
- [6] Akash Singirikonda, Serdar Kadioglu, and Karthik Uppuluri. Text2zinc : A cross-domain dataset for modeling optimization and satisfaction problems in minizinc, 2025.
- [7] Jakob Snauwaert and Mario Vanhoucke. A solution framework for multi-skilled project scheduling problems with hierarchical skills. *Journal of Scheduling*, 28 :289–310, 2025.
- [8] Yuliang Song and Eldan Cohen. Do llms understand constraint programming ? zero-shot constraint programming model generation using llms, 2025.
- [9] Stefan Szeider. Mcp-solver : Integrating language models with constraint programming systems, 2025.
- [10] Rahman Torba, Stéphane Dauzère-Pérès, Claude Yugma, Cédric Gallais, and Juliette Pouzet. Solving a real-life multi-skill resource-constrained multi-project scheduling problem. *Annals of Operations Research*, 338 :69–114, 2024.